

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA



INF 263 – DESARROLLO DE APLICACIONES MULTIMEDIA

ESTUDIANTE

BARRIONUEVO CHAVEZ ERICKA FABIOLA

13086660

LA PAZ – BOLIVIA

2026

Clasificación de Texturas en Imágenes

Resumen Ejecutivo

Se desarrolló una aplicación de clasificación supervisada capaz de diferenciar cuatro superficies dentro de una imagen (césped, tierra, asfalto y cielo). El sistema extrae características de color y textura por bloque, entrena con muestras del usuario y clasifica usando K-Nearest Neighbors con $k=3$, generando una imagen temática coloreada.

1. Objetivo

Desarrollar una aplicación capaz de diferenciar distintas superficies dentro de una imagen utilizando técnicas básicas de procesamiento digital de imágenes y principios similares a los empleados en la clasificación supervisada de imágenes satelitales.

- Césped
- Tierra
- Asfalto
- Cielo

2. Metodología Utilizada

La aplicación implementa un enfoque de clasificación supervisada estructurado en dos fases:

Fase 1 — Entrenamiento

El usuario carga una imagen y selecciona manualmente muestras representativas de cada superficie. Se extraen cuatro características por muestra:

Componente	Detalle
Media RGB	Promedio de los canales rojo, verde y azul de la región
Intensidad promedio	Valor de luminosidad media del área muestreada
Varianza	Dispersión estadística de los valores de píxel
Desviación estándar	Indicador de heterogeneidad de la textura

Las características se almacenan en una base de datos SQL Server como conjunto de entrenamiento.

Fase 2 — Clasificación

Algoritmo KNN ($k = 3$)

Para cada bloque se calcula la distancia euclidiana entre sus características y todas las muestras. Se seleccionan los 3 vecinos más cercanos y se asigna la clase mayoritaria.

Flujo del Sistema

01	Cargar imagen desde el disco
02	Seleccionar muestras de entrenamiento manualmente
03	Extraer características de color y textura por muestra
04	Almacenar muestras en la base de datos SQL Server
05	Dividir la imagen en bloques y calcular características
06	Clasificar cada bloque mediante KNN ($k = 3$)
07	Generar imagen temática coloreada por clase

3. Herramientas y Tecnologías

Componente	Detalle
Lenguaje	C#
Framework	.NET 8 · Windows Forms
Entorno	Microsoft Visual Studio 2022
Base de datos	Microsoft SQL Server
Gestión BD	SQL Server Management Studio (SSMS)
Procesamiento	System.Drawing — acceso y manipulación de píxeles
Clasificación	K-Nearest Neighbors (KNN) — $k = 3$
Características	Media RGB, intensidad, varianza, desviación estándar

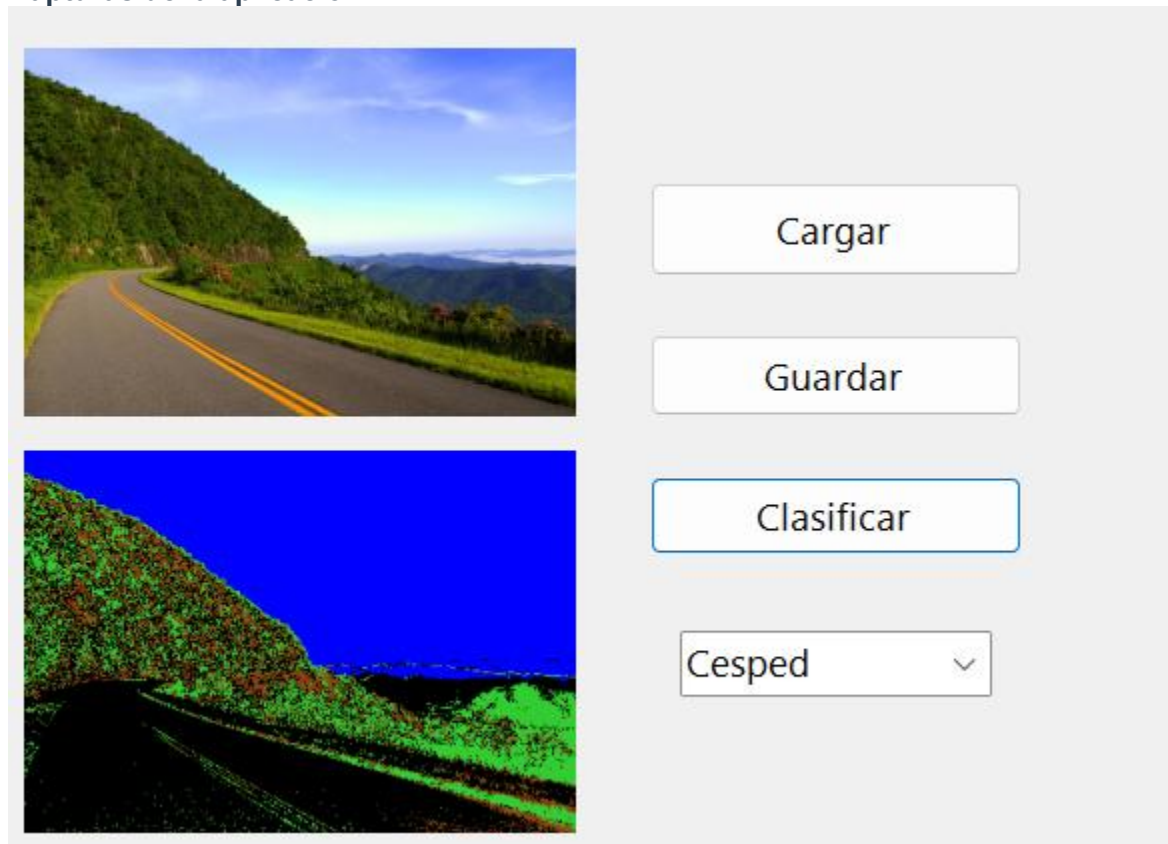
4. Resultados Obtenidos

El sistema clasificó automáticamente las cuatro superficies objetivo con los siguientes colores temáticos:

Clase	Color	Descripción
Césped	#27AE60	Superficies vegetales / campos verdes

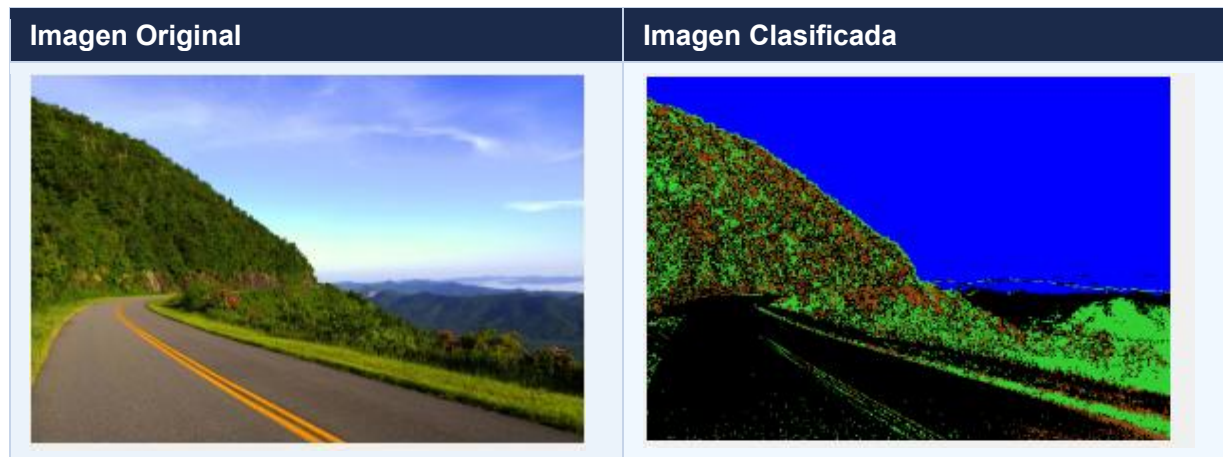
Tierra	#8B5E3C	Suelo descubierto, caminos sin pavimento
Asfalto	#555555	Carreteras, aceras, superficies grises
Cielo	#4A90D9	Zona atmosférica, nubes, horizonte

Capturas de la aplicación:



5. Comparación Visual (Antes / Después)

La imagen procesada representa cada superficie con un color específico, similar a los mapas temáticos de imágenes satelitales.



La comparación evidencia que la aplicación logra segmentar visualmente las distintas superficies presentes.

6. Conclusiones

- Se implementó exitosamente una aplicación de clasificación de texturas basada en procesamiento digital de imágenes.
- Se utilizaron características de color y textura (media RGB, intensidad, varianza y desviación estándar) para representar cada superficie.
- El algoritmo KNN permitió clasificar regiones usando muestras almacenadas en SQL Server.
- Los resultados demuestran la viabilidad de aplicar técnicas básicas de clasificación supervisada similares a las de imágenes satelitales.
- El proyecto permitió comprender la extracción de características y clasificación a nivel de píxel.

Filtro de Suavizado con Ventana 3×3

Resumen Ejecutivo

Se implementó un filtro de promedio espacial con ventana de 3×3 píxeles para reducir ruido y suavizar transiciones de color en imágenes digitales. El algoritmo reemplaza cada píxel por el promedio de sus nueve vecinos, procesando los canales RGB de forma independiente.

1. Objetivo

Desarrollar una aplicación de procesamiento digital de imágenes capaz de reducir ruido y suavizar transiciones de color mediante la implementación de un filtro de promedio con ventanas de 3×3 píxeles.

2. Metodología Utilizada

El algoritmo emplea procesamiento espacial a nivel de píxel mediante una máscara de promedio de 3×3:

Ventana 3×3

P1	P2	P3
P4	P5 (centro)	P6
P7	P8	P9

Cálculo del Promedio por Canal

$$\text{PromedioR} = (R1 + R2 + R3 + R4 + R5 + R6 + R7 + R8 + R9) / 9$$

$$\text{PromedioG} = (G1 + G2 + G3 + G4 + G5 + G6 + G7 + G8 + G9) / 9$$

$$\text{PromedioB} = (B1 + B2 + B3 + B4 + B5 + B6 + B7 + B8 + B9) / 9$$

Máscara de Convolución (1/9 en cada posición)

1 / 9	1 / 9	1 / 9
-------	-------	-------

1 / 9	1 / 9	1 / 9
1 / 9	1 / 9	1 / 9

Esta máscara distribuye igual peso a todos los píxeles de la ventana, resultando en un suavizado uniforme que atenúa variaciones bruscas de intensidad.

3. Herramientas y Tecnologías

Componente	Detalle
Lenguaje	C#
Framework	.NET 8 · Windows Forms
Entorno	Microsoft Visual Studio 2022
Librería imagen	System.Drawing — acceso a píxeles individuales
Librería UI	System.Windows.Forms
Tipo de filtro	Filtro espacial de promedio (mean filter) · 3×3
Procesamiento	Convolución 2D a nivel de píxel

4. Desarrollo del Algoritmo

01	Cargar una imagen desde el disco
02	Crear una nueva imagen vacía para almacenar el resultado
03	Recorrer todos los píxeles internos (excluyendo bordes)
04	Obtener los nueve píxeles de la ventana 3×3
05	Calcular el promedio de los componentes R, G y B por separado
06	Generar un nuevo color con los valores promediados
07	Guardar el resultado en la imagen de salida
08	Mostrar la imagen procesada en pantalla

5. Resultados Obtenidos

Se utilizó una imagen con ruido artificial para evaluar el filtro. Comparativa antes/después:

ANTES	DESPUÉS
-------	---------

X Alta presencia de ruido	✓ Ruido reducido notablemente
X Píxeles aleatorios de colores	✓ Apariencia más uniforme
X Imagen granulada	✓ Colores suavizados
X Transiciones bruscas de color	✓ Pérdida controlada de detalles

6. Comparación Visual (Antes / Después)



La comparación visual confirma que el filtro cumple correctamente con el objetivo de suavizar la imagen y reducir el ruido presente.

7. Conclusiones

- Se implementó correctamente un filtro de suavizado basado en una ventana de 3×3 píxeles con procesamiento directo a nivel de píxel.
- El filtro de promedio reduce eficazmente el ruido, generando una representación visual más uniforme.
- Los canales RGB son procesados de forma independiente, manteniendo el balance de color original.
- El algoritmo introduce una reducción controlada de detalles finos, efecto inherente al promediado.
- Los resultados evidencian el funcionamiento adecuado del filtro y la aplicación práctica de técnicas fundamentales de PDI.

Cover Multimedia: La Vaca Lola — Estilo Shakira

Resumen Ejecutivo

Se desarrolló una producción multimedia completa del cover de la canción infantil 'La Vaca Lola' utilizando la identidad artística de Shakira. El proyecto integra generación de audio con Inteligencia Artificial (MusicGen / Hugging Face), animación 3D en Unity con modelo procedural, sincronización beat-animación mediante análisis de BPM y una plataforma web funcional publicada en GitHub Pages.

1. Objetivo

Realizar una producción multimedia de la canción infantil 'La Vaca Lola' utilizando la identidad artística de Shakira, vinculando el audio generado con IA con una animación 3D en Unity, asegurando que el ritmo y el movimiento del personaje coincidan con precisión.

Objetivos específicos:

- Generar un cover musical de 'La Vaca Lola' con estilo Shakira usando Hugging Face (MusicGen).
- Construir un modelo 3D de la Vaca Lola con primitivas en Unity y accesorios temáticos.
- Sincronizar las animaciones del personaje con el ritmo detectado del audio generado.
- Publicar la plataforma web completa con formularios dinámicos y animación WebGL.
- Mantener todo el código en un repositorio GitHub con documentación de instalación.

2. Identidad Artística — Shakira

La identidad de Shakira se incorporó al proyecto en tres niveles:

Componente	Detalle
Audio / Estilo musical	Latin pop con influencia de cumbia colombiana, reggaeton y ritmo de 120 BPM. Prompt de MusicGen diseñado para emular el sonido característico de Shakira.
Visual / Personaje 3D	La Vaca Lola lleva sombrero rojo festivo, collar de perlas doradas y cascabel — accesorios de inspiración Shakira. Outfit intercambiable en 4 colores (dorado, rojo, azul, naranja).
Movimiento / Animación	Se implementó 'Shakira Hip Mode': movimiento de caderas senoidal con parámetros de intensidad y

	velocidad ajustables, activado en cada beat de la música.
--	---

3. Generación de Audio con IA

Herramienta: MusicGen (Meta AI) via Hugging Face

La música se generó en Google Colab (GPU T4) usando el modelo facebook/musicgen-small. El proceso completo:

01	Instalar transformers, audiocraft, librosa y ffmpeg en Colab
02	Cargar MusicGen Small con precisión float16 en GPU T4
03	Definir 3 prompts diferenciados: intro/base rítmica, verso-coro, puente
04	Generar ~10 segundos de audio por segmento (512 tokens)
05	Combinar segmentos con crossfade suave (estructura: Intro→Verso→Coro→Puente→Coro)
06	Analizar BPM con librosa.beat.beat_track() → ~120 BPM
07	Exportar WAV, MP3, beat_data.json y letra_sincronizada.json

Prompt de Generación — Estilo Shakira

Prompt principal (coro):

"Upbeat Latin pop children's song with Shakira style, reggaeton rhythm, acoustic guitar, upbeat tempo 120 BPM, fun and playful mood, cow bells percussion, Colombian cumbia influence, bright and cheerful"

Parámetros de generación:

max_new_tokens: 512 · guidance_scale: 3.0 · do_sample: True · temperature: 1.0

Datos de Sincronización Exportados

Componente	Detalle
beat_data.json	BPM detectado, sample rate, duración y lista de timestamps de cada beat (en segundos)
letra_sincronizada.json	Array de objetos {time, texto, duracion} para mostrar subtítulos sincronizados en Unity
vaca_lola_shakira.wav	Audio principal en 32 kHz para importar directamente en Unity
vaca_lola_shakira.mp3	Versión MP3 192kbps para el reproductor web

segmento_*.wav	Segmentos individuales (intro, verso_coro, puente) para edición adicional
----------------	---

4. Animación 3D en Unity

Modelo Procedural — VacaLolaBuilder.cs

El modelo 3D fue construido completamente con primitivas de Unity mediante código, sin necesidad de importar modelos externos. Las partes construidas son:

Parte	Descripción de construcción
Cuerpo	Esfera deformada (1.4×0.9×2.0) con 3 manchas características en color marrón oscuro
Cabeza	Esfera con hocico cilíndrico, fosas nasales, ojos con brillo, cuernos y orejas con interior rosa
Cola	4 segmentos esféricos decrecientes + pompón en color marrón
Patas (×4)	Cápsula (muslo) + cilindro (pezuña) en color marrón oscuro
Ubre	Esfera aplastada con 4 cilindros — color rosa
Sombrero	Copa cilíndrica roja + ala + banda dorada (identidad Shakira)
Collar	12 perlas dispuestas en arco — colores alternados dorado/rosa
Cascabel	Cilindro dorado colgante — identidad Shakira

Animaciones — VacaLolaCharacter.cs

Animación	Trigger	Descripción
Dance	Inicio de la música	Baile continuo: cabeceo, balanceo corporal, movimiento de cola
HipShake	Continuo (Shakira Mode)	Movimiento senoidal de caderas en X y Z con velocidad e intensidad configurables
BeatPulse	Cada beat (≈120/min)	Bounce de escala (×1.3) en 0.1s + flash de emisión en el material
TailWag	Cada 4 beats	Rotación de ±30° en la cola con interpolación suave
Jump	Cada 8 beats	Salto con arco parabólico + efecto squish al aterrizar
Moo	Al mostrar '¡Muuu!'	Cabeza se inclina -20° hacia arriba + partículas de notas musicales

Sincronización Beat-Animación — BeatSyncAnimator.cs

El script lee beat_data.json al inicio y compara el tiempo transcurrido de la canción (Time.time – songStartTime) con los timestamps de cada beat:

Lógica de sincronización:

```
while (currentTime >= beat_times[nextBeatIndex] – threshold) → OnBeatHit()
```

Acciones por frecuencia de beat:

Cada beat → bounce de escala + trigger de animador + flash visual

Cada 4 beats → animación de cola (TailWag)

Cada 8 beats → salto con squish (Jump)

Por timestamp → mostrar línea de letra (fade-in en 0.2s)

5. Herramientas y Tecnologías

Componente	Detalle
Audio IA	MusicGen Small / Medium (Meta AI) via Hugging Face Transformers
Plataforma audio	Google Colab — GPU T4 (gratuito)
Análisis de ritmo	librosa.beat.beat_track() — detección automática de BPM y timestamps
Motor 3D	Unity 2022.3 LTS — Universal Render Pipeline (URP)
Scripts Unity	C# · BeatSyncAnimator.cs, VacaLolaCharacter.cs, VacaLolaBuilder.cs
Modelo 3D	Primitivas procedurales Unity (Sphere, Cylinder, Capsule, Cube)
Publish web	Unity WebGL Build + GitHub Pages
Plataforma web	HTML5 + JavaScript vanilla — formularios dinámicos + reproductor de audio
Repositorio	GitHub con README de instalación y ejecución

6. Plataforma Web Funcional

La plataforma web publicada en GitHub Pages incluye los siguientes componentes:

Componente	Detalle
Animación 3D Unity WebGL	Canvas embebido con la animación del personaje. Controles: Play/Pausa, Reiniciar, Cambiar Outfit, Pantalla Completa.

Reproductor de Audio	Visualizador de waveform animado, control de posición (seek), indicador de BPM en tiempo real, visualizador de beats.
Formulario de Animación	Configurar nombre, outfit (4 estilos), escenario, intensidad de baile, velocidad de hip shake, activar/desactivar efectos.
Formulario de Audio	Seleccionar modelo IA, estilo artístico, BPM objetivo, duración por segmento y prompt adicional. Genera prompt listo para Colab.
Pipeline visual	Diagrama de 5 pasos: Colab → Unity → Sincronización → Build WebGL → GitHub Pages.

8. Conclusiones

- Se implementó exitosamente un cover multimedia de 'La Vaca Lola' integrando IA generativa, animación 3D y desarrollo web.
- MusicGen (Hugging Face) generó música con estilo Shakira mediante prompts de Latin pop con influencia de cumbia y reggaeton.
- El modelo 3D procedural en Unity permite construir el personaje completo sin assets externos, facilitando la portabilidad del proyecto.
- La sincronización beat-animación mediante beat_data.json logra que los movimientos del personaje coincidan con el ritmo del audio generado.
- La plataforma web publicada en GitHub Pages cumple todos los requisitos: formularios dinámicos, animación 3D WebGL y reproductor de audio.
- El repositorio GitHub documenta la instalación y ejecución completa del pipeline: Colab → Unity → Web.